

# Kubernetes 101

Conceptos iniciales

About me: Martin Perrupato, algunos me conocen por títulos como “La Muerte del Agente Mocho”, o “Bartolito en la Matrix”.  
Trabaje durante varios años en empresas nacionales y extranjeras, en roles relacionados con infraestructura de sistemas  
(Sysadmin, SRE, DevOps Engineer, Platform Engineer)

Linkedin: <https://www.linkedin.com/in/martin-perrupato/>

## Ejemplo de skills/stack de trabajo de un Infrastructure Engineer.

Scripting and hotfix- Bash / Python / Go / Typescript

CICD (Continuous Integration, Continuous Deployment)- Jenkins (custom scripts), Gitlab CI, Github

Provisioning - Infra as Code - Terraform , Crossplane, Helm + Config Connector (w/ kubernetes)

Containers Orchestration - Kubernetes (EKS, AKS ,GKE)

System administration - Linux

Troubleshooting

Monitoring

etc

Que es kubernetes? – Kubernetes es un orquestador de containers, y una plataforma para administrar aplicaciones de forma resiliente, que permite el uso de buenas prácticas de ingeniería. Permite controlar recursos para nuestras aplicaciones de forma declarativa a través de archivos yaml. Es altamente personalizable a través de custom resource definitions (lo que nos permite crear nuestras api y recursos propios).

Origen del Kubernetes (k8s) - Creado por Google y lanzado oficialmente en 2014. Su desarrollo se basó en la experiencia interna de Google con sistemas de gestión de clústeres como Borg (iniciado en 2003) y Omega, utilizando estas lecciones para crear una herramienta más accesible y escalable.

Su nombre proviene del griego para "timonel". Tras su anuncio inicial en junio de 2014, se alcanzó la versión estable 1.0 en julio de 2015, momento en el que Google donó el proyecto a la recién fundada Cloud Native Computing Foundation (CNCF).

fuelle - Ask Brave(combination of llama and qwen models).

Más info de BORG y Omega → <https://queue.acm.org/detail.cfm?id=2898444>

k8s code repository → <https://github.com/kubernetes/kubernetes>

por ejemplo el código en GO kubectl get → <https://github.com/kubernetes/kubectl/blob/master/pkg/cmd/get/get.go>

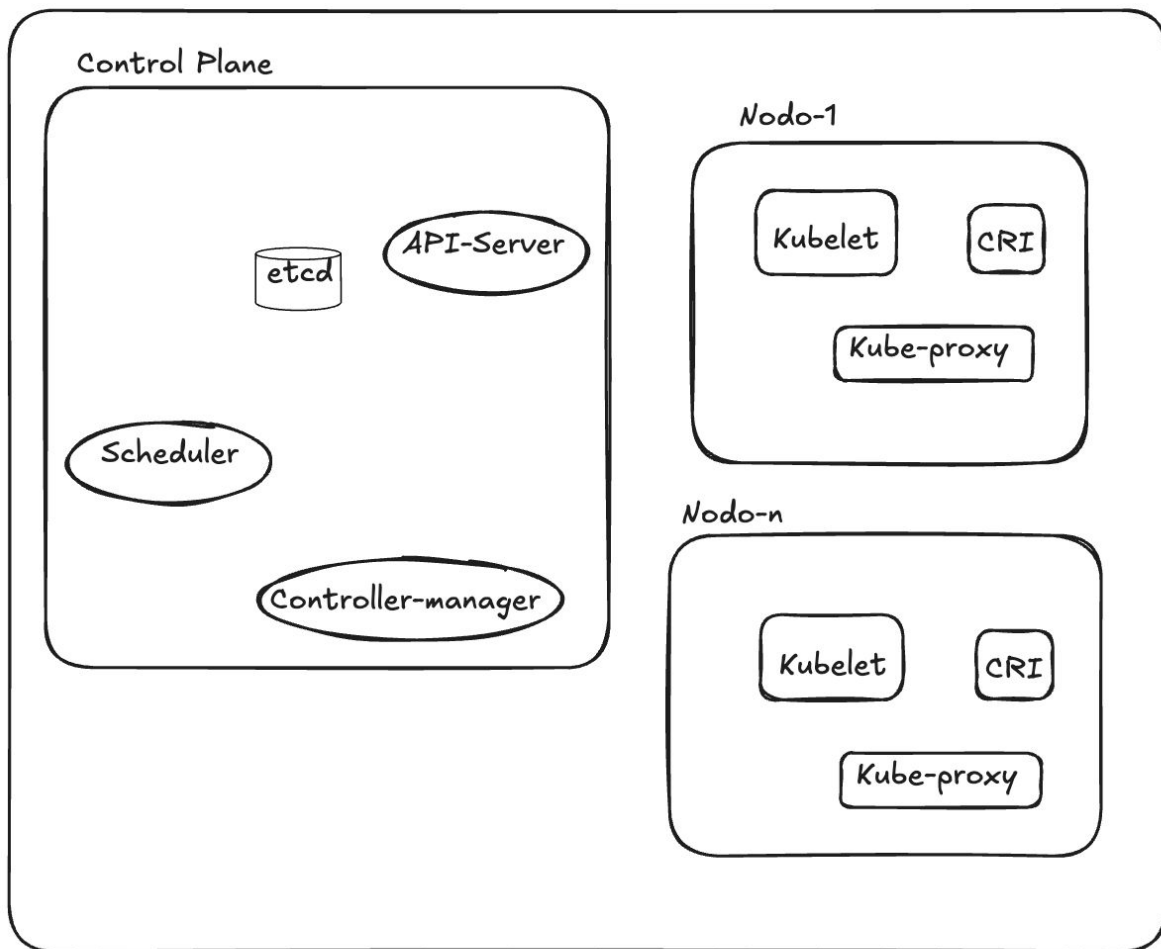
Cloud Native Computing Foundation → <https://www.cncf.io/>

```
{ Kind: Cluster }
```

Docu oficial

<https://kubernetes.io/docs/concepts/architecture/>

# Cluster de K8s



## Componentes del Control Plane

### **kube-apiserver**

El componente principal del control plane ,expone el API REST. Se encarga de validar y configurar la información para distintos objetos como los que veremos más adelante (Pods, Replica Sets, Services, etc). Por ejemplo el comando kubectl se comunica con el api-server para interactuar con el cluster.

<https://kubernetes.io/docs/reference/command-line-tools-reference/kube-apiserver/>

### **etcd**

Base de datos consistente y de alta disponibilidad de tipo key-value que almacena datos y metadata del apiserver.

<https://etcd.io/docs/v3.8/>

### **kube-scheduler**

Busca pods que no estén asignados a un nodo y los asigna a un nodo disponible.

### **kube-controller-manager**

Ejecuta los bucles de control centrales encargados de mantener el estado deseado del clúster. Funciona como un único daemon que integra múltiples controladores, cada uno de los cuales administra un tipo específico de recurso para garantizar que el estado real del clúster coincida con las especificaciones del usuario.

### **cloud-controller-manager (optional)**

Administra recursos de cloud provider(s). Similar al kube-controller-manager pero con lógica para interactuar con recursos del cloud provider.



## Componentes del nodo

### **kubelet**

Asegura que los pods y sus containers estén en ejecución.

### **kube-proxy**

Mantiene las reglas de networking en el nodo.

### **Container runtime**

Software responsable de ejecutar containers (ejemplo docker).

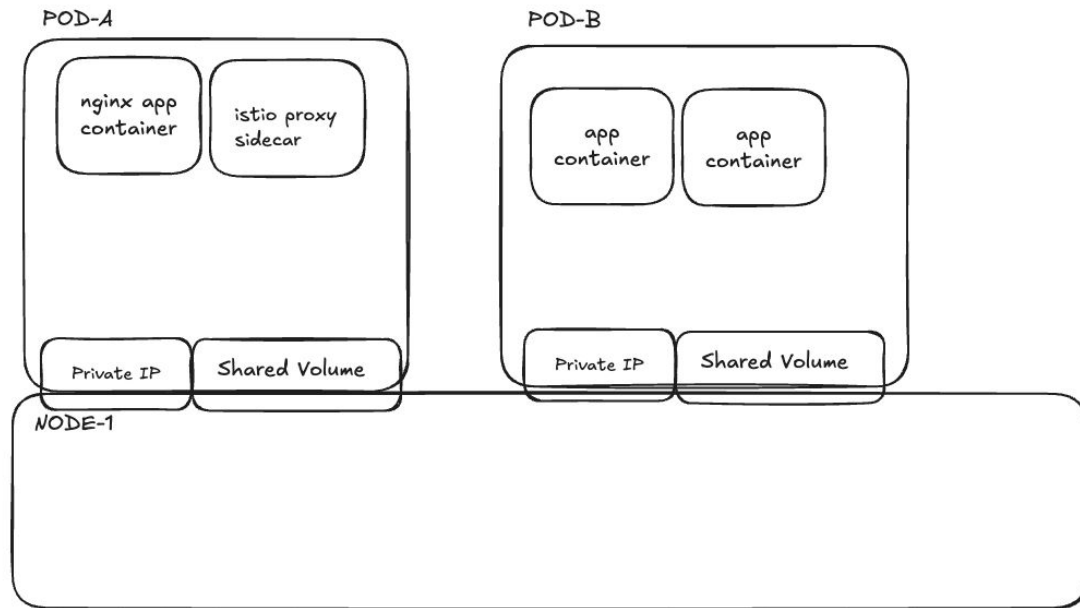
`{ Kind:Pod }` - Pod is a collection of containers that can run on a host. This resource is created by clients and scheduled onto hosts.

El Pod es una abstracción de Kubernetes que nos permite definir uno o más containers que pueden compartir recursos.

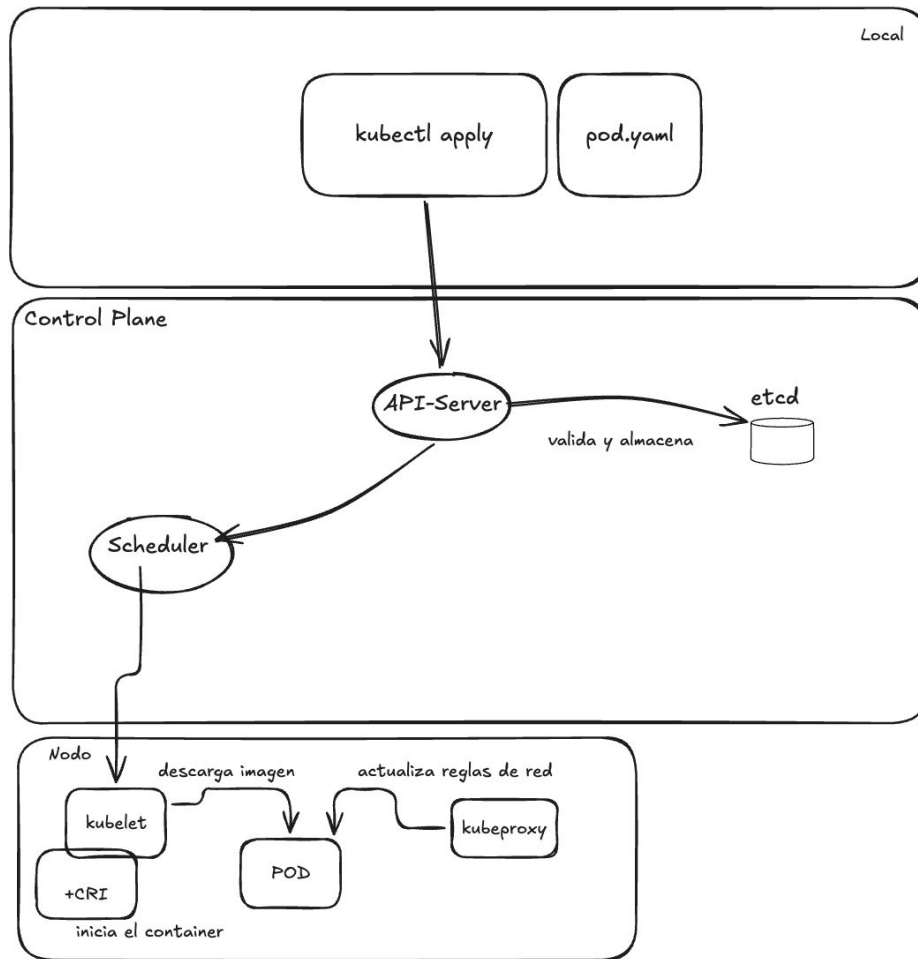
Dentro de los recursos que se pueden compartir entre containers los más habituales son:

- Volumes
- Networking (ip del nodo)

Limitaciones del Pod - Nos permite definir e implementar recursos para una aplicación pero no hay nada a este nivel que nos garantice mantener la configuración definida.



camino de un kubectl apply ("inspirado" en un diagrama de devopscube)



`{ Kind: Deployment }` - Deployment enables declarative updates for Pods and ReplicaSets.

Un deployment es un recurso de kubernetes que nos permite definir el estado deseado para una aplicación.

Algunas cosas que podemos definir en el deployment:

- número de Pods
- image de la app
- rollout/rollback (volver a una versión previa)
- restart/recovery (recuperación ante fallos)

```
{ Kind: Service }
```

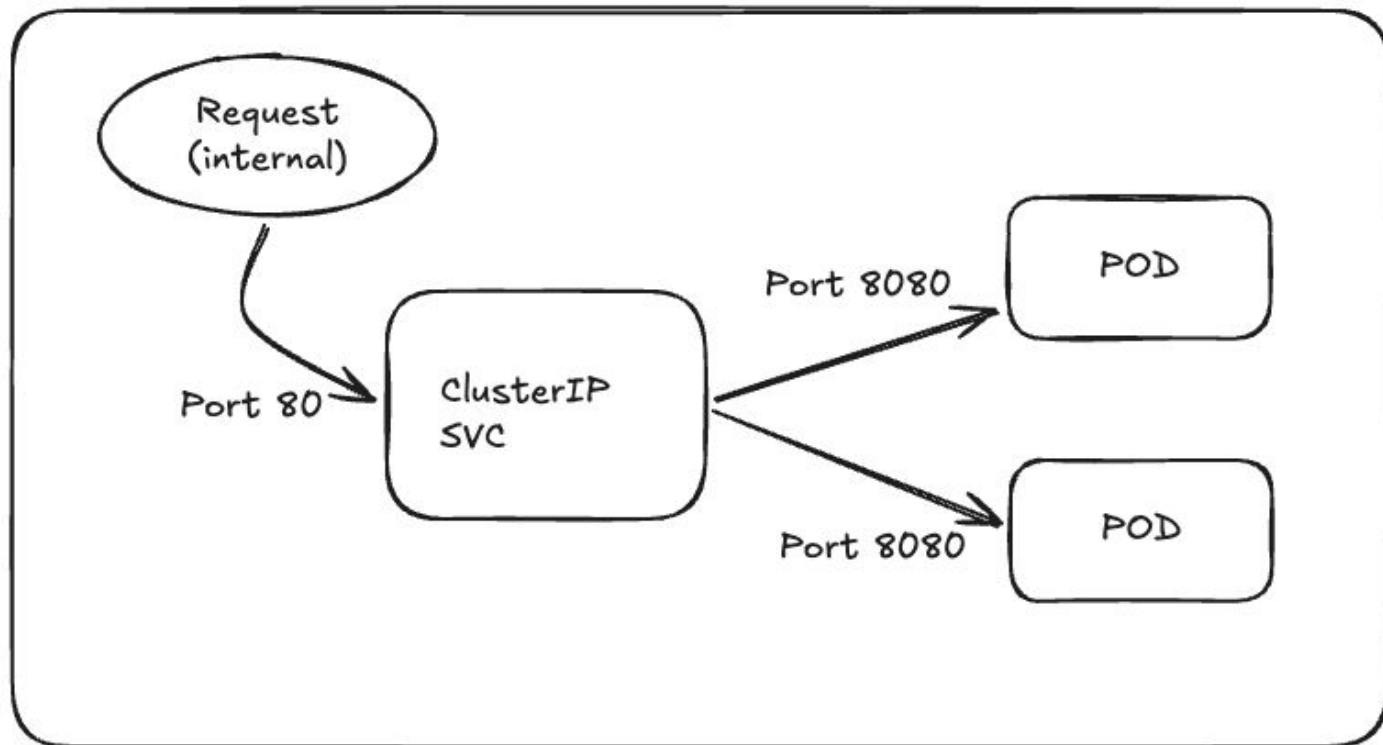
Service is a named abstraction of software service (for example, mysql) consisting of local port (for example 3306) that the proxy listens on, and the selector that determines which pods will answer requests sent through the proxy.

“Un Deployment crea y administra Pods, mientras que un Service les da una forma de comunicarse.”

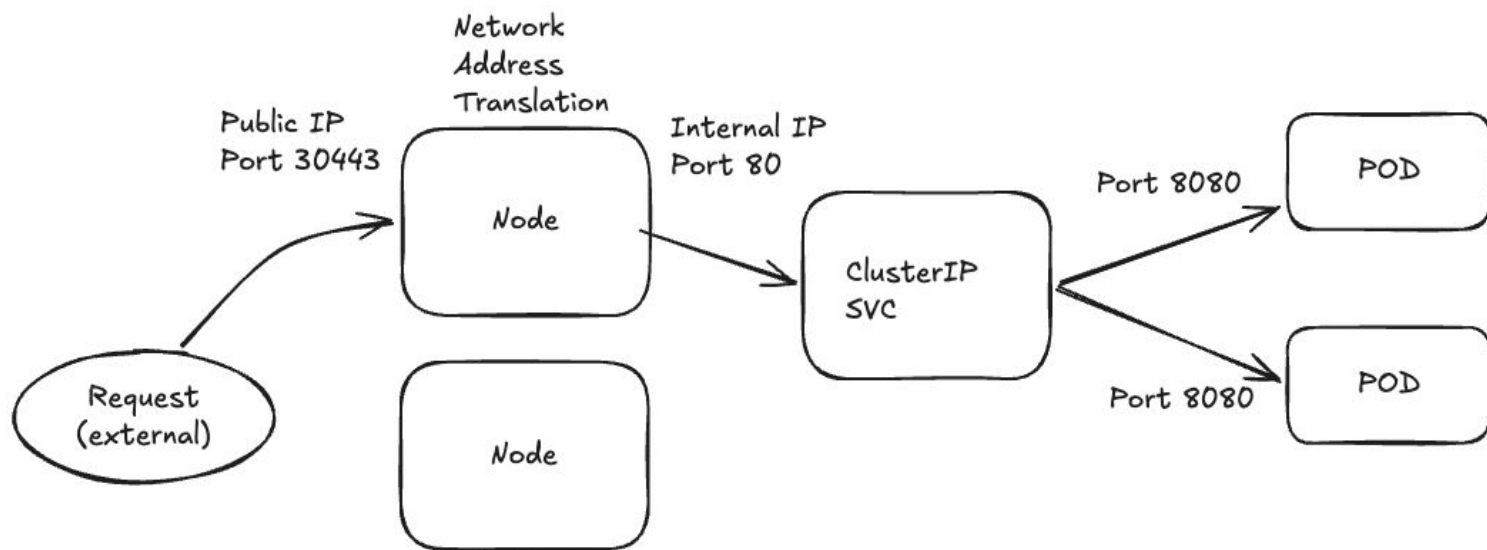
Algunas de las formas más comunes de exponer un servicio:

- ClusterIP (default) - Expone el cluster de forma privada por una ip privada del cluster. Solo se accede desde el mismo cluster.
- NodePort - Expone el servicio en el mismo puerto de cada nodo seleccionado del clúster mediante NAT. Permite acceder al servicio desde fuera del clúster.
- LoadBalancer - Crea un Load Balancer en el cloud actual (si está soportado) y asigna una ip pública.

### ClusterIP Service (internal traffic)

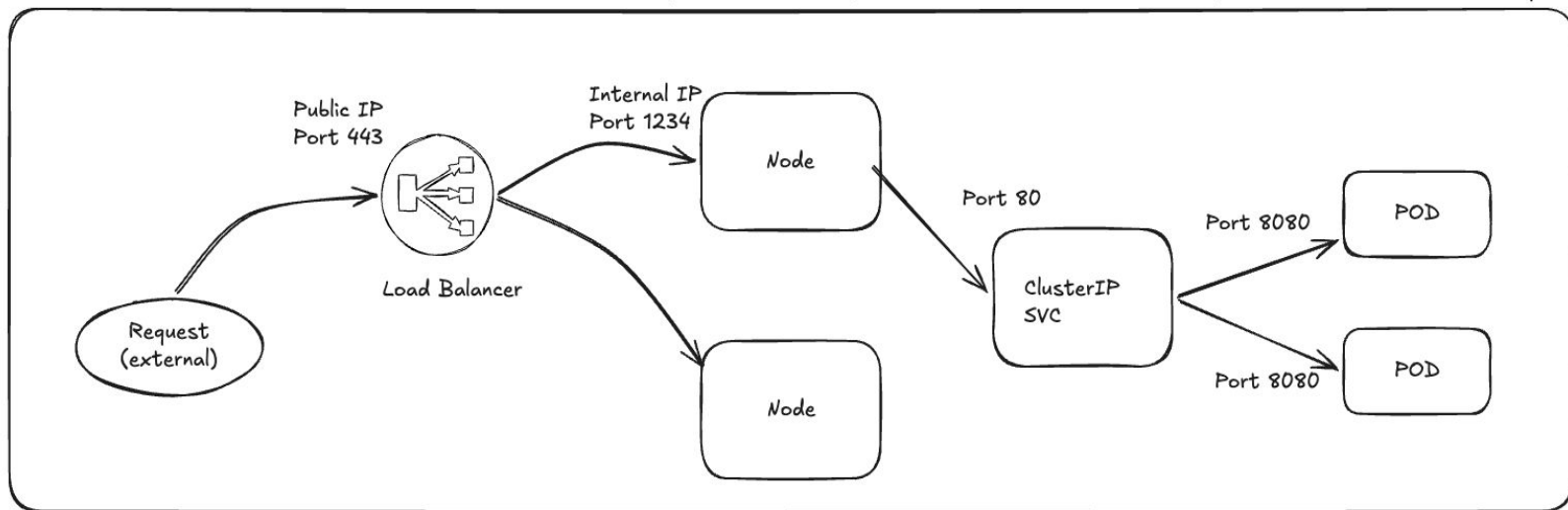


Node Port Service (Allow external traffic via ports of the node) - Superset ClusterIP





LoadBalancer Service - allows external traffic via load balancer, implements and manage load balancer resource into cloud provider kubernetes cluster. Superset NodePort



DEMO - Revisar los conceptos explicados en un cluster de kubernetes local.

Para profundizar...

Docu oficial

<https://kubernetes.io/docs/tutorials/kubernetes-basics/>

Libro

“Kubernetes: Up and Running, 3rd Edition” by Brendan Burns, Joe Beda, Kelsey Hightower, Lachlan Evenson